Milestone 1: Project Initiation, Core Security & Gateway Entry Portals

1. Architectural Paradigm & System Design

SkillSphere Learning Nexus is architected on a high-availability full-stack model built to bridge education and recruitment pipelines.

- **Backend Component Model:** Java 17 and Spring Boot (3.2.5) act as the enterprise core. It enforces clean separation of concerns through controllers (REST resources), services (business layer logic), repositories (JPA mappings to MySQL), and configs (security configuration policies).
- **Frontend Single Page Application (SPA):** Powered by React 19 and Vite. Utilizes a Glassmorphic layout scheme with CSS variables for animations, transitions, and component fluid rendering.
- **API Routing Gateway:** All client calls flow through the routing controls, entering the Spring Security filter validation chains, checking JWT claim lists, and routing requests to appropriate API handlers.

2. Implemented Features: User Gateways & Entry Portals

- **Landing Page Interface:** Created LandingPage.jsx displaying dynamic routes for Student, Recruiter, and Admin portals. Displays modern styling utilizing animations and hover scale transformations.
- **Login & Registration Portals:** Developed form fields checking password criteria, email patterns, and role classifications (Student, Recruiter, Admin) before processing registrations.
- **Role-Based Access Routing:** Set up routing paths protecting sections, redirecting unauthorized navigation attempts to login pages based on active security roles.
- **System Management Configurations:** Programmed CORS configuration beans, database pool drivers (HikariCP parameters), and connection variables in application.properties.

3. Implemented Features: Authentication & Authorization Security

- **JWT Stateless Security:** Built SecurityConfig.java configuring stateless filters. Configured custom JWT authentication providers checking incoming header tokens.
- **JWT Token Generation & Validation:** Formulated JwtTokenProvider.java. Generates 512-bit signed tokens carrying username, expiration date, and user role authorization tags.
- **Google OAuth 2.0 Integration:** Implemented backend GoogleOAuth2Service using GoogleIdTokenVerifier. Validates client-side OAuth tokens and enables one-click user creation.

4. Configuration Parameters & Dependencies

- **Project Dependencies (pom.xml):** Packaged spring-boot-starter-web, spring-boot-starter-security, spring-boot-starter-data-jpa, mysql-connector-j, and io.jsonwebtoken (jjwt-api, jjwt-impl).
- **Client Dependencies (package.json):** Configured react-router-dom, axios, and lucide-react.
- **Database Schemas:** Generated User, Role, and user_roles table records, setting indexing constraints on username and email database columns.

5. Deliverables & Technical Verification

- **[X]** React 19 and Spring Boot 3.2.5 projects fully initialized.
- **[X]** Database schema definitions and relationship configurations mapped.
- **[X]** Responsive Glassmorphic landing page and login screens verified.
- **[X]** Custom JwtAuthenticationFilter and TokenProvider successfully deployed.
- **[X]** Google OAuth authentication client-server flows validated.